

TECHNICAL ARCHITECTURE DOCUMENT

AI & Tech News

A Google News–style discovery and aggregation platform for AI & technology, with a Netflix-inspired reading UI.

Node.js / Express

Vercel Serverless

RSS Aggregation

Claude AI Summaries

No Database

Prepared: July 20, 2026

Repository: `ai_tech_news`

Branch: `claude/news-aggregation-platform-fow2iw`

1. What This Platform Is

AI & Tech News is a news **discovery and aggregation** platform, not a publisher. It pulls headlines from 36 RSS/Atom feeds across major tech press, mainstream tech desks, and AI-focused outlets, groups and ranks them, attaches a short AI-written (or extractive) summary, and links every card straight back to the original publisher.

Core legal/product rule: the platform never stores, republishes, or displays full article bodies. Every card shows only a headline, a 40–100 word summary, and source attribution — always with a direct outbound link to the publisher.

Why no database?

News has no long-term state worth persisting — it is fetched, scored, and re-fetched every 60 seconds. The entire "database" is an in-memory JavaScript array (`src/store.js`), rebuilt from live feeds on an interval. This removes an entire category of infrastructure (schema migrations, connection pooling, ORM) for a workload that doesn't need it.

2. Technology Stack

Layer	Choice	Why
Runtime	Node.js $\geq$ 18, CommonJS	No build step, no bundler, no TypeScript compile — files run as-is.
Backend (local)	Express (<code>server.js</code>)	Serves <code>public/</code> statically and exposes the JSON API for <code>npm start</code> development.
Backend (production)	Vercel Serverless Functions (<code>api/*.js</code>)	Each file is one endpoint; both runtimes share the exact same <code>src/store.js</code> logic.
Data source	36 RSS/Atom feeds via <code>rss-parser</code>	No database — an in-memory store refreshed on a timer.
AI summarization	Claude (<code>claude-opus-4-8</code>) via <code>@anthropic-ai/sdk</code>	Optional. Falls back automatically to a clean extractive summary when no API key is configured.
Frontend	Vanilla HTML / CSS / JS in <code>public/</code>	Zero frameworks, zero frontend dependencies — a hand-written <code>el()</code> DOM helper builds the UI.
Hosting	Vercel	Auto-deploys every push to the default branch; static files + functions in one deploy.

Dependencies (package.json)

Package	Purpose
<code>express</code>	Local dev HTTP server + static file serving
<code>compression</code>	Gzip responses in local dev
<code>rss-parser</code>	Parses RSS/Atom XML from all 36 sources
<code>@anthropic-ai/sdk</code>	Claude API client for AI summaries (only instantiated if a key is present)
<code>@vercel/functions</code>	Provides <code>waitUntil()</code> so background refresh work survives after the response is sent on Vercel

3. High-Level Architecture



4. The News Pipeline (`src/store.js`)

Every refresh cycle runs the same six steps over all 36 sources in parallel:

1. **Fetch** — each source is requested with a 7-second timeout and a real browser User-Agent (several publishers 403 non-browser clients). Malformed XML ampersands are sanitized before parsing.
2. **Normalize** — each entry becomes a compliant "news card": `id` (SHA-1 hash of the canonicalized link), `title`, `link`, `sourceName`, `publishedAt`, a short `excerpt` (stripped HTML, capped at 150 words — used only as summarization input, never shown as body text), and an `image` pulled from media tags or an `og:image` lookup.
3. **Classify** (`src/lib/classify.js`) — keyword-rule engine assigns one of 10 categories (AI Models, Research, Startups & Funding, Policy & Regulation, Security, Hardware & Chips, Robotics, Cloud & Enterprise, Consumer Tech, Open Source), tags up to 4 companies from a 29-company keyword list, and a region.
4. **Score** (`src/lib/score.js`) — a weighted signal score (breaking-news keywords, funding/M&A/launch language, source authority weight, presence of an image) with an 8-hour exponential recency half-life, and a 0.25× penalty for newsletter/roundup/deals-style posts so they never lead the homepage.
5. **Dedupe & merge** — new results are merged into the existing set by ID and by a normalized title key (first 10 words, punctuation stripped) so the same story from two outlets doesn't appear twice. Critically, **a source that fails to fetch does not erase its previous stories** — items are only dropped once they exceed the 7-day max age.
6. **Summarize** (`src/lib/summarize.js`) — see Section 5.

Why merge instead of replace: an earlier version of the store replaced the whole item list on every refresh. A single slow or rate-limited source would then make its entire homepage section vanish until the next successful fetch. The current store always re-adds previously known items that a fetch round didn't return, as long as they're still within the 7-day age window.

5. AI Summaries — and the Fallback That's Currently Active

This is the part you asked about directly. Summarization lives entirely in `src/lib/summarize.js`, and it is designed to degrade gracefully:

With `ANTHROPIC_API_KEY` set

A Claude client is constructed at startup. New/un-summarized items are sent to `claude-opus-4-8` in batches of 10, with a strict JSON-schema response: `{ id, summary }`. The system prompt asks for a neutral, factual, 40–70 word summary — what happened, who's involved, why it matters, no opinion or clickbait.

Without a key (current state)

`client` stays `null`, `aiEnabled()` returns `false`, and every item instead gets an **extractive summary**: the first few sentences of the feed's own excerpt, trimmed to 50–90 words. No external API call is made at all.

Current state of this deployment: there is no `ANTHROPIC_API_KEY` configured (checked: no `.env` file, and the variable is unset).

This means the platform is running entirely on the **extractive fallback** right now — every summary on the site is the first 1–3 sentences of the publisher's own RSS excerpt, not an LLM-generated summary. The site is fully functional either way; nothing breaks, nothing is blocked. Adding the key later in Vercel → Project → Settings → Environment Variables (or a local `.env`) turns AI summaries back on with no code changes.

Why this matters for compliance

- Only the headline and the short RSS excerpt are ever sent to Claude — never the full article body, which the platform never even fetches.
- Both paths cap the summary at 100 words and run it through the same downstream rendering — the UI cannot tell the difference between an AI summary and an extractive one, and doesn't need to.
- A failed or refused Claude call for a batch is caught and silently leaves those items on their extractive summary rather than erroring the whole refresh.

Why summarization is background-only in production

A Claude batch call can take 30+ seconds. The one hard rule of the request path (see Section 6) is that a visitor's request is never blocked on it — summaries are attached in the background, and a story may render with its extractive summary for a few seconds before quietly upgrading to the AI version on the next poll.

6. The Request Path — Why the Site Never Feels Slow

This is the most carefully engineered part of the codebase, encoded as the "non-negotiable invariants" in the project's own CLAUDE.md. The rule: **a request is never blocked on feed fetching**, except for the one boot request that guarantees a fresh first paint.

Data is found in this priority order

1. In-memory store (state.items)

2. CDN copy of /api/snapshot

3. data/seed.json bundled in the deploy

– instant, the common case

– ~100ms, used by a cold serverless instance

– instant, last-resort bootstrap

Any real feed refresh always runs strictly in the BACKGROUND

(setInterval locally · waitUntil() on Vercel) – never in front

of a response.

Two request modes

Mode	Used by	Behavior
<code>ensureReadyForRequest()</code> (api/_shared.js)	Every normal request: <code>/api/feed</code> , <code>/api/rows</code> , plain <code>/api/home</code>	Answers from whatever's already in memory (or hydrates from snapshot/seed if the store is empty). If the data is stale, it kicks off a background refresh via <code>waitUntil</code> and returns immediately — the visitor never waits on it.
<code>ensureFresh()</code> (src/store.js)	Only <code>/api/home?fresh=1</code> — the very first call <code>boot()</code> makes in <code>public/app.js</code>	Deliberately blocks (up to a 12-second budget) on one summary-free refresh, so the first thing a new visitor sees is today's news, not a stale seed. It skips AI summarization entirely so it can't get stuck behind a 30-second Claude call.

Serverless cold-start bootstrap chain

Cold function instance starts, state.items is empty

↓

Try: hydrate from https://<deployment>/api/snapshot (2s timeout)

| fails / empty

↓

Try: load data/seed.json bundled into the deployment (instant)

|

↓

Store now has data → answer the request → refresh in background

Caching headers add a second layer on top: `Cache-Control: s-maxage=60, stale-while-revalidate=300` means most repeat requests to a popular deployment never even reach a function — Vercel's CDN edge answers them directly.

7. API Endpoints

Endpoint	Handler	Purpose
GET /api/home	api/home.js	The one call the homepage boots from: hero + trending rail + category rails + categories + first page of the grid, in one response. <code>?fresh=1</code> triggers the blocking freshness guarantee described above.
GET /api/feed	api/feed.js	Paginated grid for infinite scroll and category filtering (<code>?page=&limit=&category=</code>).
GET /api/rows	api/rows.js	Just the Netflix-style rails: billboard hero, "Trending Now", and one rail per category.
GET /api/categories	api/categories.js	Fixed taxonomy with live per-category counts.
GET /api/snapshot	api/snapshot.js	Full in-memory store dump — what cold serverless instances hydrate from instead of re-fetching all 36 feeds.
GET /api/status	api/status.js	Health check: 200 while the store has data, 503 if every source is failing (this is what an uptime monitor should page on), plus whether AI summaries are currently enabled.

8. Frontend Behavior (`public/`)

Zero frameworks — `app.js` (718 lines) builds the entire UI with a small `el()` DOM-construction helper, driven by the JSON from `/api/home`.

- **Boot sequence:** `boot()` calls `/api/home?fresh=1` for the first paint, then polls plain `/api/home` periodically.
- **Never yanks the reader:** when a poll finds new stories, the UI does not silently re-render underneath someone reading — it shows a red "New stories" pill (`offerUpdate()`) that the visitor clicks to apply the update on their own terms.
- **Fixed section order:** rails and category chips always render in the same canonical order (`SECTION_ORDER` in the store) so the homepage layout never reshuffles between refreshes.
- **Outbound links only:** every card links directly to the publisher with `rel="noopener external"`, opens in a new tab, and shows a small "leaving this site" toast — reinforcing that this is a discovery layer, not a copy of the article.
- **Cookie/storage consent gate:** a blocking gate on first visit (localStorage key `cookie-consent`) that the visitor must resolve (Accept all / Necessary only) before interacting with the rest of the page. Google Analytics (gtag.js) only loads after "Accept all."
- **Notification permission prompt:** after the consent gate is resolved, the site asks for browser notification permission (re-prompting on each visit until the visitor explicitly allows or denies) and, once granted, fires a native OS notification when a new top story lands.
- **Theming:** light theme by default, dark mode via a toggle, entirely through CSS custom properties (no CSS-in-JS, no theme library).

9. File-by-File Map

Path	Role
server.js	Express app for local dev — security headers, static file serving, all API routes, starts the background refresh loop.
api/_shared.js	Shared serverless plumbing: <code>ensureReadyForRequest()</code> , cache headers, the 202 "warming" response for the rare cold-start-with-no-data case.
api/home.js, feed.js, rows.js, categories.js, snapshot.js, status.js	One Vercel serverless function per endpoint (see Section 7).
src/store.js	The core: fetch/normalize/classify/score/dedupe/merge/summarize, the bootstrap hydration chain, and every read accessor (<code>getRows</code> , <code>getFeed</code> , <code>getCategories</code> , <code>getSnapshot</code> , <code>getHealth</code>).
src/config/sources.js	The 36 RSS/Atom sources with name, feed URL, homepage, and an editorial-authority weight (5–9) that feeds hero/trending scoring.
src/lib/classify.js	Keyword-rule engine → category (10-way taxonomy), company tags (29 companies), region (7 regions).
src/lib/score.js	Importance scoring for hero/trending selection: keyword signals, source weight, image bonus, roundup penalty, recency half-life decay.
src/lib/summarize.js	Claude batch summarization with JSON-schema output, plus the extractive fallback described in Section 5.
src/lib/text.js	HTML-stripping and sentence-splitting utilities shared by normalization and summarization.
data/seed.json	Bundled news snapshot used as the last-resort bootstrap so even the very first request after a fresh deploy is instant. Regenerated via <code>npm run build:seed</code> .
scripts/build-seed.js	Regenerates <code>data/seed.json</code> from a live refresh.
public/index.html, app.js, styles.css	The entire frontend — no build step, no bundler.
vercel.json	Sets serverless function <code>maxDuration</code> and related Vercel project config.

10. Deployment

Pushing to the default branch (`claude/news-aggregation-platform-fow2iw`) auto-deploys on Vercel. `public/` is served as static assets; every file in `api/` becomes an independent serverless function. Running `npm run build:seed` before a big deploy refreshes the bundled bootstrap news, though it isn't required — the store self-updates in the background within seconds of the first real request either way.

Environment variables

Variable	Default	Effect
<code>ANTHROPIC_API_KEY</code>	<code>unset</code>	Optional. Enables Claude AI summaries. Currently unset in this project — see Section 5.

SUMMARY_MODEL	claude-opus-4-8	Which Claude model summarizes batches, when a key is present.
REFRESH_INTERVAL_MS	60000	How often the background loop re-fetches all 36 feeds.
PORT	3000	Local dev server port only.

Generated from a direct read of the current repository state (server.js, api/*.js, src/store.js, src/lib/*.js, src/config/sources.js, public/app.js, package.json) on July 20, 2026. No ANTHROPIC_API_KEY was found configured, confirming the platform is presently operating on its extractive summary fallback.